FILE: C:\HCL_TRAINING\server\src\services\skillExtractor\skillExtractor.js

LINES: 89 | SIZE: 2.5 KB | LANGUAGE: JS

MODULE: 500+ Skill Parser with Word Boundary Lookaheads

```
const { SKILL_TAXONOMY } = require('./taxonomy');

/**
 * Escapes regex special characters
 */
function escapeRegex(str) {
  return str.replace(/[.*+?^${}()|[\]\\]/g, '\\$&');
}

/**
 * Creates a smart boundary pattern that supports C#, C++, Node.js, and normal words
 */
function createSkillPattern(term) {
  const escaped = escapeRegex(term);
  const startBound = /^[a-zA-Z0-9]/.test(term) ? '\\b' : '(?:^|\\s)';
  const endBound = /[a-zA-Z0-9]$/.test(term) ? '\\b' : '(?=$|\\s|[,.!?;:\\(\\)])';
  return new RegExp(`${startBound}${escaped}${endBound}`, 'i');
}

/**
 * Extracts skills and supporting evidence from text
 */
function extractSkillsFromText(text) {
  const detectedSkills = [];
  const sentences = text.split(/(?<=[.?!])\s+|\n+/).filter((s) => s.trim().length > 10);

  for (const [category, skillsList] of Object.entries(SKILL_TAXONOMY)) {
    for (const skillObj of skillsList) {
      const { name, aliases } = skillObj;
      const allNames = [name, ...aliases];

      let isMatched = false;
      let matchedTerm = '';
      let evidenceExcerpt = '';

      for (const term of allNames) {
        const regex = createSkillPattern(term);
        if (regex.test(text)) {
          isMatched = true;
          matchedTerm = term;

          // Find the best sentence providing evidence
          for (const s of sentences) {
            if (regex.test(s)) {
              evidenceExcerpt = s.trim();
              if (evidenceExcerpt.length > 160) {
                evidenceExcerpt = evidenceExcerpt.substring(0, 157) + '...';
              }
              break;
            }
          }
          break;
        }
      }

      if (isMatched) {
        detectedSkills.push({
          name: name,
          category: category,
          matchedAlias: matchedTerm !== name ? matchedTerm : undefined,
          evidence: evidenceExcerpt || `Demonstrated capability in ${matchedTerm} in can
> didate experience profile`,
          confidence: 0.95,
          yearsOfExperience: estimateSkillYears(name, text),
        });
      }
```

```
66 |       }
67 |     }
68 |
69 |     return detectedSkills;
70 |   }
71 |
72 |   /**
73 |    * Heuristically estimates years of experience for a skill based on text context
74 |    */
75 |   function estimateSkillYears(skillName, text) {
76 |     const pattern = new RegExp(`(\\d+)\\+?\\s*years?(?:\\s+of)?(?:\\s+experience)?(?:\\s+(
   >   ?:with|in|using))?\\s+${escapeRegex(skillName)}`, 'i');
77 |     const match = text.match(pattern);
78 |     if (match) {
79 |       const yrs = parseInt(match[1], 10);
80 |       return Math.min(yrs, 20);
81 |     }
82 |     return 3;
83 |   }
84 |
85 |   module.exports = {
86 |     extractSkillsFromText,
87 |     createSkillPattern,
88 |   };
89 |
```